¿La mejora se mantiene al mover la matriz?

Los experimentos anteriores medían el kernel de factorización. Sin embargo, una aplicación puede comenzar con la matriz A completa en un proceso y necesitar que el factor L vuelva completo al mismo proceso. En ese caso, repartir y reunir los datos también forman parte del costo útil.

Caso de uso que medimos

Fase	MPI	Secuencial
Entrada	El rango 0 empaqueta las columnas cíclicas y ejecuta MPI_Scatterv.	Copia A al arreglo que se sobrescribirá.
Factorización	Right con vector local o left distribuido.	Right-looking con vector contiguo.
Salida	MPI_Gatherv y desempaquetado del factor completo en rango 0.	El factor ya está completo en el arreglo local.

Qué queda fuera

La reserva de memoria, la generación de A, la comprobación del resultado y el arranque de los procesos MPI quedan fuera. El lanzamiento puede pagarse una vez si una aplicación mantiene trabajadores activos, o en cada problema si crea un nuevo trabajo; por eso debe estudiarse por separado.

Tiempo total observado

$$T_{\text{total}} = T_{\text{entrada}} + T_{\text{factor}} + T_{\text{salida}}$$

Cada proceso registra marcas consecutivas. Para reportar cada fase y el total tomamos el máximo entre procesos. Debido a que los máximos de las fases podrían pertenecer a rangos distintos, la suma de los máximos por fase no tiene que coincidir exactamente con el máximo total.

Esta definición representa un flujo centralizado de entrada y salida. No representa aplicaciones donde la matriz ya nace distribuida o el factor continúa utilizándose de forma distribuida.

Protocolo de la comparación completa

Decisión	Valor
Algoritmos	Right MPI cíclico con vector local; left MPI cíclico; right secuencial con vector.
Tamaños y procesos	n=64,128,256,512,1024; P=1,2,3,4.
Repeticiones	Un calentamiento y 12 mediciones por algoritmo y configuración. Las seis permutaciones del orden se repiten dos veces.
Entrada	La misma matriz SPD, generada en C con semilla 2026, comienza completa en el rango 0.
Salida	El factor se reúne completo en rango 0 y se compara con la referencia secuencial.
Entorno	WSL2, Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0, -O3, double, sin BLAS.

Comparación contemporánea

La referencia secuencial se ejecuta dentro del mismo lanzamiento y ocupa todas las posiciones posibles del orden. Para cada n y P comparamos MPI con esa referencia contemporánea. No dividimos por tiempos de PDF anteriores.

Dos preguntas distintas

$$S_{\text{kernel}} = \frac{\widetilde{T}_{\text{factor, seq}}}{\widetilde{T}_{\text{factor, MPI}}}, \qquad S_{\text{total}} = \frac{\widetilde{T}_{\text{total, seq}}}{\widetilde{T}_{\text{total, MPI}}}$$

La primera indica cuánto acelera Cholesky una vez repartidos los datos. La segunda decide si conviene el flujo completo descrito. Una optimización puede mejorar el kernel y aun perder cuando la matriz es pequeña.

Las barras de los gráficos muestran Q1-Q3 de 12 repeticiones. Son medidas descriptivas de dispersión, no intervalos de confianza.

Resultado: el cruce práctico se desplaza

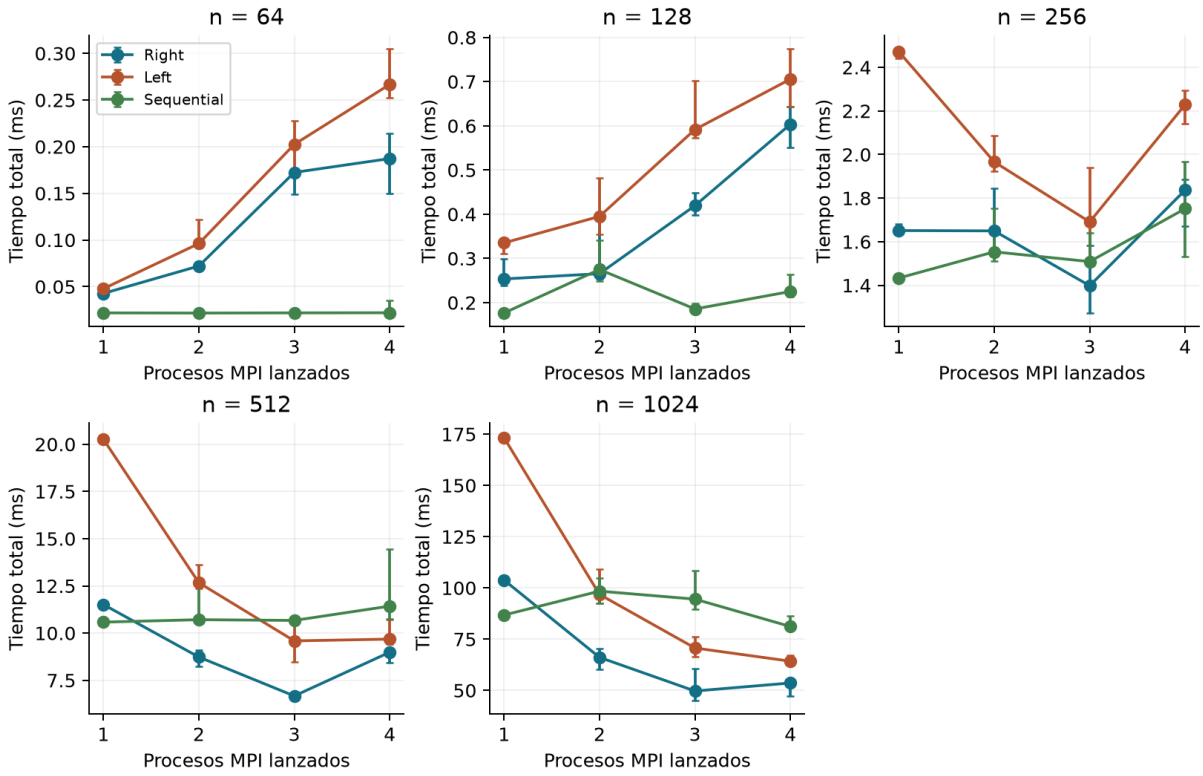


Figura 1. Tiempo completo: entrada, factorización y salida. Cada panel usa su propia escala. La referencia secuencial se repite para cada P porque se mide dentro del mismo lanzamiento.

n	Mejor P≥2	Sec. (ms)	Right total (ms)	Aceleración
64	2	0,022	0,072	0,30
128	2	0,275	0,266	1,04
256	3	1,509	1,398	1,08
512	3	10,670	6,671	1,60
1024	3	94,523	49,699	1,90

Qué cambia respecto del kernel

n=64: la coordinación y las copias dominan.
n=128: la mejor configuración paralela apenas iguala al secuencial.
n=256: right con P=3 mejora solo 1,08x, por debajo del margen práctico de 10 % usado en la etapa 4.
n=512 y n=1024: la ventaja total ya es clara: 1,60x y 1,90x con P=3.

En esta grilla, **512 es el primer tamaño con una mejora total clara**. Es una observación de esta campaña, no un umbral universal.

Qué costo agregan la entrada y la salida

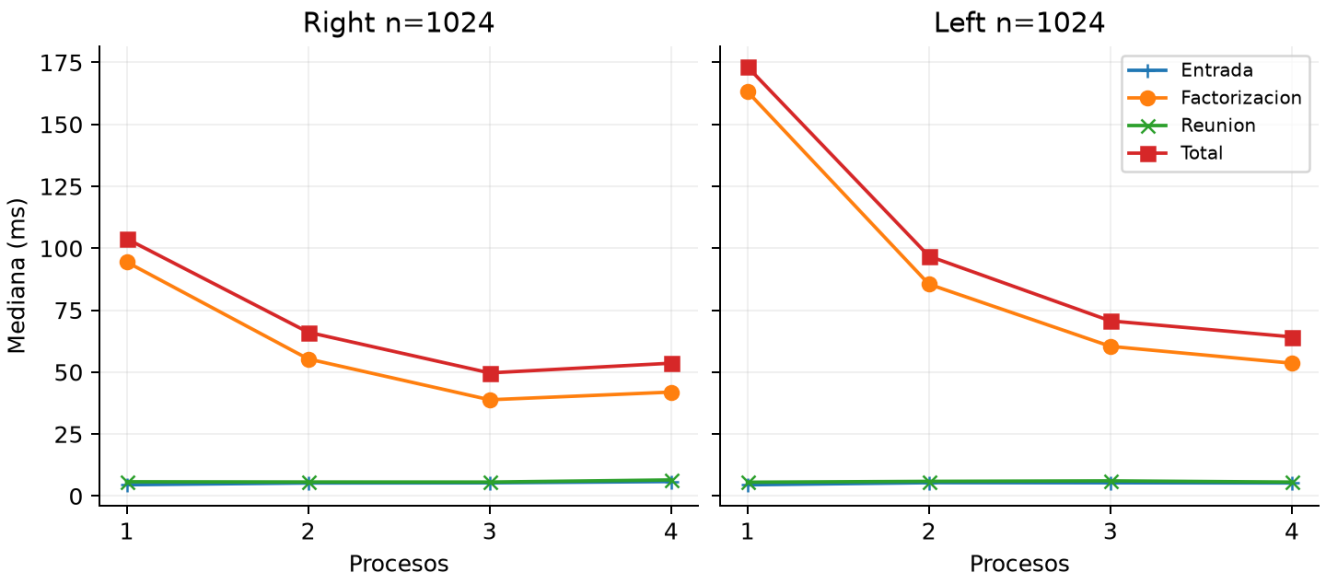


Figura 2. Medianas de cada fase y del total para n=1024. Las líneas de fases no deben sumarse como si provinieran del mismo rango y repetición.

Right con n=1024 y P=3

Entrada	Factorización	Salida	Total
5,323 ms	38,848 ms	5,630 ms	49,699 ms

El kernel right mejora 2,41x frente al secuencial. Al incluir entrada y salida, la aceleración queda en 1,90x. Los movimientos de datos reducen la ganancia, pero no la eliminan en este tamaño.

Right mantiene la ventaja frente a left

Con n=1024 y P=3, left tarda 70,678 ms en total frente a 49,699 ms de right. Como ambos usan el mismo reparto y las mismas fases externas, la diferencia continúa concentrándose en la factorización: 60,417 ms para left y 38,848 ms para right.

Una limitación visible

La entrada y la salida permanecen alrededor de cinco a seis milisegundos cada una para n=1024 y no disminuyen al aumentar P en esta máquina. El rango 0 empaqueta y desempaqueta en forma secuencial; ese diseño se convertirá en cuello de botella al escalar.

Una aplicación real podría evitar esas conversiones generando o leyendo directamente el formato distribuido, o conservar el factor distribuido. Eso cambiaría el caso de uso y debe medirse como otro experimento.

Conclusión y siguiente etapa

Decisión actual

Right-looking MPI cíclico con vector local sigue siendo la variante elegida. Fue más rápido que left en el kernel y en el flujo completo. Con matrices pequeñas conviene la versión secuencial; para n=512 y n=1024, tres procesos dieron una mejora total clara en esta campaña.

Qué afirmamos y qué no

Afirmamos resultados sobre una entrada y salida centralizadas en WSL. No sabemos todavía cómo cambia el costo dentro de un nodo del clúster ni, especialmente, entre nodos. Tampoco medimos lectura desde archivo, escritura, lanzamiento del trabajo o varias factorizaciones consecutivas.

Validación

Se aceptaron 720 observaciones. El residuo relativo máximo fue 1,2131×10^-15 y la diferencia relativa máxima frente al factor secuencial fue 2,6578×10^-15. Las pruebas funcionales y los sanitizadores con P=4 terminaron correctamente; las huellas de las fuentes medidas coinciden.

Siguiente experimento cuando tengamos el clúster

Repetiremos right secuencial y right MPI con P=1,2,3,4 dentro de un nodo. Luego compararemos una asignación equivalente entre nodos, registrando nodos, procesos por nodo, afinidad, compilador y biblioteca MPI. Mediremos entrada, kernel, salida y total con el mismo criterio.

Si la red hace dominar la comunicación, el siguiente cambio razonable será agrupar operaciones por bloques para reducir la frecuencia de mensajes y aprovechar BLAS. No conviene implementar esa complejidad antes de observar el problema en el clúster.

Archivo en desde_cero	Contenido
src/compare_mpi_total.c	Cronometra fases, reúne y valida.
scripts/comparar_tiempo_total.sh	Campaña completa reproducible.
scripts/analizar_tiempo_total.py	Estadísticas y gráficos por fase.
resultados/tiempo_total_20260922/	CSV, protocolo, afinidad, huellas y fuentes.